Tugas Praktikum Sesi 5
Pemrograman Mobile



Dosen Pengampu: Jefry Sunupurwa Asri, S.Kom, M.Kom

Disusun Oleh:

Bagas Yoas Sibagariang

(20230801254)

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS ILMU KOMPUTER

UNIVERSITAS ESA UNGGUL

TANGERANG

2026

1. **Langkah Praktikum**

a. SharedPreferences (Pengaturan Sederhana)

Pasang paket, lalu buat aplikasi yang menyimpan nama, pilihan mode gelap, dan hitungan berapa kali aplikasi dibuka:

```
→  praktikum_5 git:(main) ✗ flutter pub add shared_preferences
   Woah! You appear to be trying to run flutter as root.
   We strongly recommend running the flutter tool without superuser privileges.
  /
Resolving dependencies...
Downloading packages... (2.8s)
  cupertino_icons 1.0.9 (2.0.0 available)
+ ffi 2.2.0
+ file 7.0.1
+ flutter_web_plugins 0.0.0 from sdk flutter
  material_color_utilities 0.13.0 (0.13.1 available)
  meta 1.18.3 (1.19.0 available)
+ path_provider_linux 2.2.2
+ path_provider_platform_interface 2.1.3
+ path_provider_windows 2.3.0
+ platform 3.2.0
+ plugin_platform_interface 2.1.8
+ shared_preferences 2.5.6
+ shared_preferences_android 2.4.28
+ shared_preferences_foundation 2.5.7
+ shared_preferences_linux 2.4.1
+ shared_preferences_platform_interface 2.4.2
+ shared_preferences_web 2.4.3
+ shared_preferences_windows 2.4.1
  test_api 0.7.12 (0.7.15 available)
+ web 1.1.1
+ xdg_directories 1.1.0
Changed 17 dependencies!
4 packages have newer versions incompatible with dependency constraints.
Try `flutter pub outdated` for more information.
→  praktikum_5 git:(main) ✗
```

```dart
import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Praktikum 5',
      home: const PengaturanPage(),
    );
  }
}

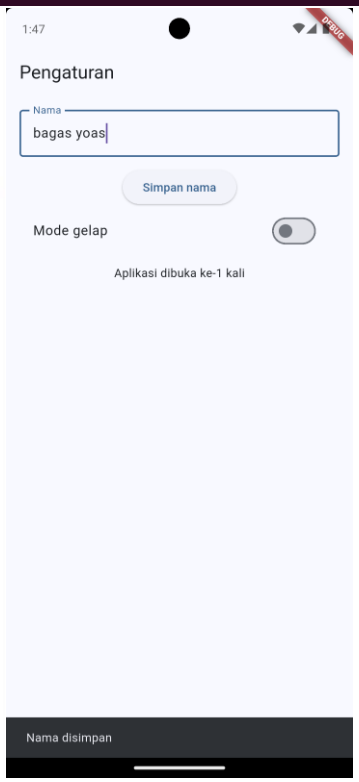
class PengaturanPage extends StatefulWidget {
  const PengaturanPage({super.key});
  @override
  State<PengaturanPage> createState() => _PengaturanPageState();
}

class _PengaturanPageState extends State<PengaturanPage> {
  final _controller = TextEditingController();
  bool _gelap = false;
  int _bukaKe = 0;
  @override
  void initState() {
    super.initState();
    _muat();
  }
  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  Future<void> _muat() async {
    final prefs = await SharedPreferences.getInstance();
    if (!mounted) return;
    setState(() {
      _controller.text = prefs.getString('nama') ?? '';
      _gelap = prefs.getBool('gelap') ?? false;
      _bukaKe = (prefs.getInt('bukaKe') ?? 0) + 1;
    });
    await prefs.setInt('bukaKe', _bukaKe);
  }

  Future<void> _simpanNama() async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setString('nama', _controller.text.trim());
    if (!mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Nama disimpan')),
    );
  }

  Future<void> _ubahTema(bool nilai) async {
    setState(() => _gelap = nilai);
    final prefs = await SharedPreferences.getInstance();
    await prefs.setBool('gelap', nilai);
  }
  @override
  Widget build(BuildContext context) {
    return Theme(
      data: ThemeData(
        brightness: _gelap ? Brightness.dark : Brightness.light,
        colorSchemeSeed: Colors.blue,
        useMaterial3: true,
      ),
      child: Scaffold(
        appBar: AppBar(title: const Text('Pengaturan')),
        body: Padding(
          padding: const EdgeInsets.all(16),
          child: Column(
            children: [
              TextField(
                controller: _controller,
                decoration: const InputDecoration(
                  labelText: 'Nama',
                  border: OutlineInputBorder(),
                ),
              ),
              const SizedBox(height: 12),
              ElevatedButton(
                onPressed: _simpanNama,
                child: const Text('Simpan nama'),
              ),
              SwitchListTile(
                title: const Text('Mode gelap'),
                value: _gelap,
                onChanged: _ubahTema,
              ),
              const SizedBox(height: 12),
              Text('Aplikasi dibuka ke-$_bukaKe kali'),
            ],
          ),
        ),
      ),
    );
  }
}
```



b. Menyiapkan SQLite

Buat proyek baru, lalu pasang paket:

flutter pub add sqflite path

```
→  praktikum_5 git:(main) ✗ flutter pub add sqflite path
   Woah! You appear to be trying to run flutter as root.
   We strongly recommend running the flutter tool without superuser privileges.
  /
Resolving dependencies...
Downloading packages...
  cupertino_icons 1.0.9 (2.0.0 available)
  material_color_utilities 0.13.0 (0.13.1 available)
  meta 1.18.3 (1.19.0 available)
  path 1.9.1 (from transitive dependency to direct dependency)
+ sqflite 2.4.4+1
+ sqflite_android 2.4.4
+ sqflite_common 2.5.13+1
+ sqflite_darwin 2.4.4
+ sqflite_platform_interface 2.4.2
+ synchronized 3.4.2
  test_api 0.7.12 (0.7.15 available)
Changed 7 dependencies!
4 packages have newer versions incompatible with dependency constraints.
Try `flutter pub outdated` for more information.
→  praktikum_5 git:(main) ✗
```

Buat model Catatan dan kelas pembantu database DbHelper. Letakkan di main.dart (atau berkas terpisah), dengan import berikut di atas:

```dart
import 'package:flutter/material.dart';
import 'package:path/path.dart' as p;
import 'package:sqflite/sqflite.dart';

void main() => runApp(const MyApp());
class Catatan {
  final int? id;
  final String judul;
  final String isi;

  const Catatan({this.id, required this.judul, required this.isi});
  Map<String, Object?> toMap() => {'id': id, 'judul': judul, 'isi': isi};
  factory Catatan.fromMap(Map<String, Object?> m) => Catatan(
    id: m['id'] as int,
    judul: m['judul'] as String,
    isi: m['isi'] as String,
  );
}
class DbHelper {
  static Database? _db;
  static Future<Database> get database async {
    if (_db != null) return _db!;
    final path = p.join(await getDatabasesPath(), 'catatan.db');
    _db = await openDatabase(
      path,
      version: 1,
      onCreate: (db, version) {
        return db.execute(
          'CREATE TABLE catatan('
              'id INTEGER PRIMARY KEY AUTOINCREMENT, '
              'judul TEXT NOT NULL, '
              'isi TEXT NOT NULL)',
        );
      },
    );
    return _db!;
  }

  static Future<int> tambah(Catatan c) async {
    final db = await database;
    return db.insert('catatan', c.toMap());
  }
  static Future<List<Catatan>> semua() async {
    final db = await database;
    final rows = await db.query('catatan', orderBy: 'id DESC');
    return rows.map(Catatan.fromMap).toList();
  }
  static Future<int> ubah(Catatan c) async {
    final db = await database;
    return db.update('catatan', c.toMap(), where: 'id = ?', whereArgs: [c.id]);
  }
  static Future<int> hapus(int id) async {
    final db = await database;
    return db.delete('catatan', where: 'id = ?', whereArgs: [id]);
  }
}
```

c. Halaman Daftar Catatan (Read dan Delete)

Ubah home: pada MyApp menjadi const CatatanPage(), lalu tambahkan:

```dart
class MyApp extends StatelessWidget {
  const MyApp({super.key});
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Praktikum 5',
      theme: ThemeData(colorSchemeSeed: Colors.blue, useMaterial3: true),
      home: const CatatanPage(),
    );
  }
}
// Bagian C: Daftar Catatan (Read dan Delete)
class CatatanPage extends StatefulWidget {
  const CatatanPage({super.key});
  @override
  State<CatatanPage> createState() => _CatatanPageState();
}

class _CatatanPageState extends State<CatatanPage> {
  late Future<List<Catatan>> _future;
  @override
  void initState() {
    super.initState();
    _future = DbHelper.semua();
  }
  void _muat() {
    setState(() {
      _future = DbHelper.semua();
    });
  }
  Future<void> _buka([Catatan? catatan]) async {
    await Navigator.push(
      context,
      MaterialPageRoute(builder: (_) => FormCatatanPage(catatan: catatan)),
    );
    if (!mounted) return;
    _muat();
  }
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Catatan Saya')),
      body: FutureBuilder<List<Catatan>>(
        future: _future,
        builder: (context, snapshot) {
          if (snapshot.connectionState == ConnectionState.waiting) {
            return const Center(child: CircularProgressIndicator());
          }
          if (snapshot.hasError) {
            return Center(child: Text('Galat: ${snapshot.error}'));
          }
          final data = snapshot.data!;
          if (data.isEmpty) {
            return const Center(child: Text('Belum ada catatan'));
          }
          return ListView.builder(
            itemCount: data.length,
            itemBuilder: (context, i) {
              final c = data[i];
              return ListTile(
                title: Text(c.judul),
                subtitle:
                Text(c.isi, maxLines: 2, overflow: TextOverflow.ellipsis),
                onTap: () => _buka(c),
                trailing: IconButton(
                  icon: const Icon(Icons.delete),
                  onPressed: () async {
                    await DbHelper.hapus(c.id!);
                    _muat();
                  },
                ),
              );
            },
          );
        },
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: () => _buka(),
        child: const Icon(Icons.add),
      ),
    );
  }
}
```

d. Form Tambah dan Ubah Catatan (Create dan Update)

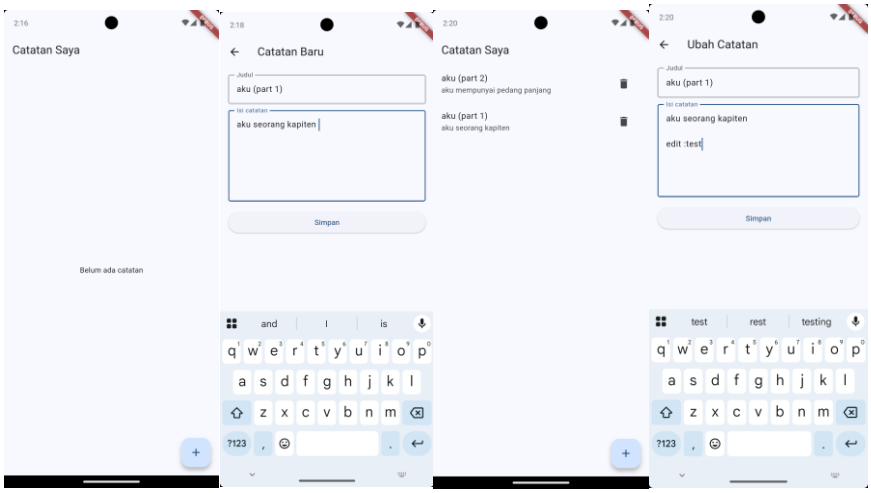
Satu halaman form dipakai untuk dua keperluan: bila catatan kosong berarti tambah, bila terisi berarti ubah.

```dart
class FormCatatanPage extends StatefulWidget {
  final Catatan? catatan;
  const FormCatatanPage({super.key, this.catatan});
  @override
  State<FormCatatanPage> createState() => _FormCatatanPageState();
}

class _FormCatatanPageState extends State<FormCatatanPage> {
  final _formKey = GlobalKey<FormState>();
  late final TextEditingController _judul;
  late final TextEditingController _isi;
  @override
  void initState() {
    super.initState();
    _judul = TextEditingController(text: widget.catatan?.judul ?? '');
    _isi = TextEditingController(text: widget.catatan?.isi ?? '');
  }
  @override
  void dispose() {
    _judul.dispose();
    _isi.dispose();
    super.dispose();
  }

  Future<void> _simpan() async {
    if (!_formKey.currentState!.validate()) return;
    final c = Catatan(
      id: widget.catatan?.id,
      judul: _judul.text.trim(),
      isi: _isi.text.trim(),
    );
    if (widget.catatan == null) {
      await DbHelper.tambah(c);
    } else {
      await DbHelper.ubah(c);
    }
    if (!mounted) return;
    Navigator.pop(context);
  }
  @override
  Widget build(BuildContext context) {
    final baru = widget.catatan == null;
    return Scaffold(
      appBar: AppBar(title: Text(baru ? 'Catatan Baru' : 'Ubah Catatan')),
      body: Form(
        key: _formKey,
        child: ListView(
          padding: const EdgeInsets.all(16),
          children: [
            TextFormField(
              controller: _judul,
              decoration: const InputDecoration(
                labelText: 'Judul',
                border: OutlineInputBorder(),
              ),
              validator: (v) =>
              (v == null || v.trim().isEmpty) ? 'Judul wajib diisi' : null,
            ),
            const SizedBox(height: 12),
            TextFormField(
              controller: _isi,
              maxLines: 6,
              decoration: const InputDecoration(
                labelText: 'Isi catatan',
                border: OutlineInputBorder(),
              ),
              validator: (v) =>
              (v == null || v.trim().isEmpty) ? 'Isi wajib diisi' : null,
            ),
            const SizedBox(height: 16),
            ElevatedButton(onPressed: _simpan, child: const Text('Simpan')),
          ],
        ),
      ),
    );
  }
}
```

Hasil bagian BCD:



2. Latihan Mandiri

1. Tampilkan AlertDialog konfirmasi "Hapus catatan ini?" sebelum data dihapus.

```
Future<void> _konfirmasiHapus(Catatan c) async {
  final ya = await showDialog<bool>(
    context: context,
    builder: (ctx) => AlertDialog(
      title: const Text('Hapus catatan ini?'),
      content: Text(c.judul),
      actions: [
        TextButton(
          onPressed: () => Navigator.pop(ctx, false),
          child: const Text('Batal'),
        ), TextButton
        TextButton(
          onPressed: () => Navigator.pop(ctx, true),
          child: const Text('Hapus'),
        ), TextButton
      ],
    ), AlertDialog
  );
  if (ya != true) return;       // Batal atau dialog dit
  await DbHelper.hapus(c.id!);
  if (!mounted) return;
  _muat();
}
```

```
return ListView.builder(
  itemCount: data.length,
  itemBuilder: (context, i) {
    final c = data[i];
    return ListTile(
      title: Text(c.judul),
      subtitle:
      Text(c.isi, maxLines: 2, overflow: TextOverflow.ellipsis),
      onTap: () => _buka(c),
      trailing: IconButton(
        icon: const Icon(Icons.delete),
        onPressed: () => _konfirmasiHapus(c),
        // onPressed: () async {
        //   await DbHelper.hapus(c.id!);
        //   _muat();
        // },
      ), IconButton
```



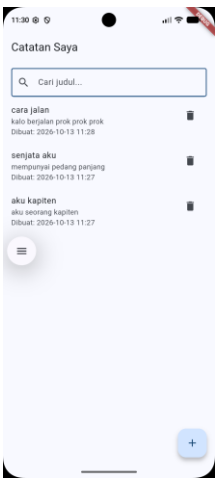
2. Tambahkan kolom pencarian yang memfilter daftar memakai where: 'judul LIKE ?', whereArgs: ['%$kata%'].

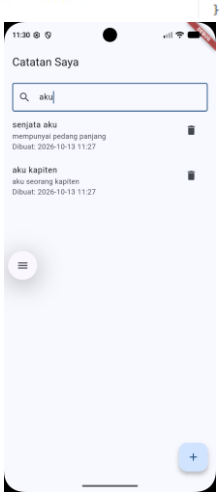
```
static Future<List<Catatan>> cari({String kata = ''}) async {
  final db = await database;
  final rows = await db.query(
    'catatan',
    where: kata.isEmpty ? null : 'judul LIKE ?',
    whereArgs: kata.isEmpty ? null : ['%$kata%'],
    orderBy: 'id DESC',
  );
  return rows.map(Catatan.fromMap).toList();
}
```

```
class _CatatanPageState extends State<CatatanPage> {
  late Future<List<Catatan>> _future;
  String _kata = '';

  @override
  void initState() {
    super.initState();
    _future = DbHelper.cari(kata: _kata);
  }
  void _muat() {
    setState(() {
      _future = DbHelper.cari(kata: _kata);
    });
  }
}
```

```
body: Column(
  children: [
    Padding(
      padding: const EdgeInsets.fromLTRB(16, 12, 16, 4),
      child: TextField(
        decoration: const InputDecoration(
          hintText: 'Cari judul...',
          prefixIcon: Icon(Icons.search),
          border: OutlineInputBorder(),
        ), InputDecoration
        onChanged: (v) {
          _kata = v.trim();
          _muat();
        },
      ), TextField
    ), Padding
    Expanded(
      child: FutureBuilder<List<Catatan>>(
        future: _future,
        builder: (context, snapshot) {
          if (snapshot.connectionState == ConnectionState.waiting) {
            return const Center(child: CircularProgressIndicator());
          }
          if (snapshot.hasError) {
            return Center(child: Text('Galat: ${snapshot.error}'));
          }
          final data = snapshot.data!;
          if (data.isEmpty) {
            return Center(
              child: Text(
                _kata.isEmpty
                    ? 'Belum ada catatan'
                    : 'Catatan tidak ditemukan',
              ), Text
            ); Center
          }
```





3. Tambahkan kolom dibuat (tanggal dibuat). Naikkan version menjadi 2 dan tambahkan onUpgrade berisi ALTER TABLE catatan ADD COLUMN dibuat TEXT. Amati apa yang terjadi bila version tidak dinaikkan.

```dart
class Catatan {
  final int? id;
  final String judul;
  final String isi;
  final String? dibuat;

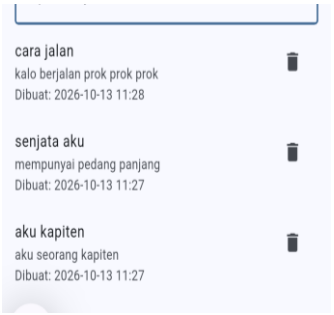
  const Catatan({this.id, required this.judul, required this.isi, this.dibuat});
  Map<String, Object?> toMap() => {'id': id, 'judul': judul, 'isi': isi, 'dibuat': dibuat};
  factory Catatan.fromMap(Map<String, Object?> m) => Catatan(
    id: m['id'] as int,
    judul: m['judul'] as String,
    isi: m['isi'] as String,
    dibuat: m['dibuat'] as String?,
  );
}
```

```dart
_db = await openDatabase(
  path,
  version: 2,
  onCreate: (db, version) {
    return db.execute(
      'CREATE TABLE catatan('
          'id INTEGER PRIMARY KEY AUTOINCREMENT, '
          'judul TEXT NOT NULL, '
          'isi TEXT NOT NULL, '
          'dibuat TEXT)',
    );
  },
  onUpgrade: (db, oldVersion, newVersion) async {
    if (oldVersion < 2) {
      await db.execute('ALTER TABLE catatan ADD COLUMN dibuat TEXT');
    }
  },
);
return _db!;
}
```

```dart
Future<void> _simpan() async {
  if (!_formKey.currentState!.validate()) return;
  final baru = widget.catatan == null;
  final c = Catatan(
    id: widget.catatan?.id,
    judul: _judul.text.trim(),
    isi: _isi.text.trim(),
    dibuat: baru ? DateTime.now().toIso8601String() : widget.catatan!.di
  ); Catatan
  if (baru) {
    await DbHelper.tambah(c);
  } else {
    await DbHelper.ubah(c);
  }
  if (!mounted) return;
  Navigator.pop(context);
}
@override
```

```dart
// tambah format tanggal
String formatTanggal(String? iso) {
  if (iso == null || iso.length < 16) return '-';
  return iso.substring(0, 16).replaceAll('T', ' ');
}
```

```dart
      maxLines: 3,
      overflow: TextOverflow.ellipsis,
    ), Text
    isThreeLine: true,
    onTap: () => _buka(c),
```

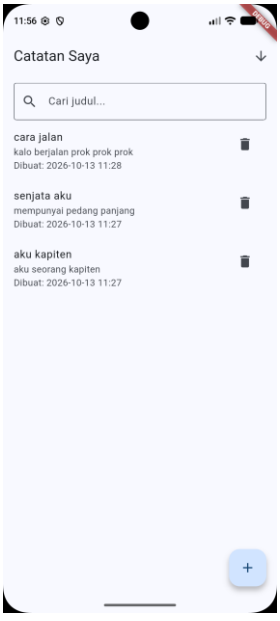


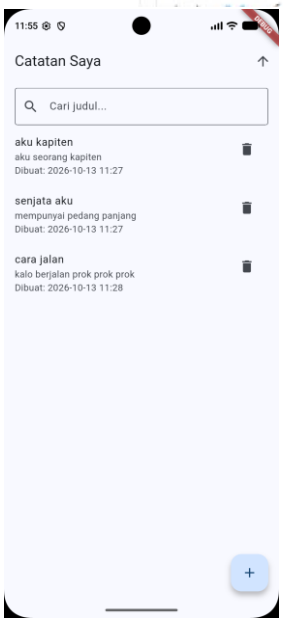
4. Simpan pilihan urutan (terbaru/terlama) dengan shared_preferences dan terapkan pada orderBy

```dart
@override
void initState() {
  super.initState();
  _future = _ambil();
}
// Baca pilihan urutan dari shared_preferences, lalu ambil data.
Future<List<Catatan>> _ambil() async {
  final prefs = await SharedPreferences.getInstance();
  final terbaru = prefs.getBool('terbaru') ?? true;
  if (mounted && terbaru != _terbaru) {
    setState(() => _terbaru = terbaru);
  }
  return DbHelper.cari(kata: _kata, terbaru: terbaru);
}

void _muat() {
  setState(() {
    _future = _ambil();
  });
}
// Simpan pilihan urutan, lalu muat ulang.
Future<void> _ubahUrutan() async {
  final prefs = await SharedPreferences.getInstance();
  final baru = !_terbaru;
  await prefs.setBool('terbaru', baru);
  if (!mounted) return;
  setState(() => _terbaru = baru);
  _muat();
}
```

```dart
static Future<List<Catatan>> cari({String kata = '', bool terbaru = true,}) async {
  final db = await database;
  final rows = await db.query(
    'catatan',
    where: kata.isEmpty ? null : 'judul LIKE ?',
    whereArgs: kata.isEmpty ? null : ['%$kata%'],
    orderBy: terbaru ? 'id DESC' : 'id ASC',
  );
  return rows.map(Catatan.fromMap).toList();
}
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Catatan Saya'),
      actions: [
        IconButton(
        tooltip: _terbaru ? 'Urutan: terbaru' : 'Urutan: terlama',
          icon: Icon(_terbaru ? Icons.arrow_downward : Icons.arrow_upward),
          onPressed: _ubahUrutan,
        ), IconButton
      ],
    ), AppBar
```





3. **Tugas**

Buat aplikasi Pencatat Pengeluaran dengan ketentuan:

- Tabel pengeluaran: id, nama, jumlah (integer), kategori, dan tanggal.
- CRUD lengkap: daftar, tambah, ubah, hapus, dengan form berisi validasi (nama wajib, jumlah angka lebih dari 0, kategori dari dropdown).
- Menampilkan total pengeluaran di halaman utama (dihitung di Dart atau memakai SELECT SUM(jumlah)).
- Menyimpan satu pengaturan (misalnya mode gelap atau kategori filter terakhir) dengan shared_preferences.
- Data harus tetap ada setelah aplikasi ditutup dan dibuka kembali.
- Kumpulkan: tangkapan layar (daftar, form, dan bukti data bertahan setelah restart) serta berkas kode (atau tautan repositori)

Hasil Tugas (di Folder Tugas_5)

